

The payroll setup screen, and why you can trust what it tells you

Slice books-25 — August 21, 2026

Michael, you told me something about payroll that I have kept in front of me the whole time I was building this:

"Payroll has always been something I've feared because I have had no reliable way of knowing I'm doing it right."

That sentence is a description of a missing instrument, not a description of a missing skill. You know what a W-4 is. You know what withholding is. What you have never had is a machine that will tell you, out loud and before you commit, whether the thing you just entered is complete and correct. Sage will take anything you type and save it with a cheerful little confirmation, and then you find out nine months later on a tax form.

So this slice is one screen. It is the screen where an employee's W-4, I-9 and pay information get entered, and it is built around a single promise: it will not let you save something it cannot stand behind, and when it stops you it will tell you exactly which field is the problem and why that field matters.

The short version

There is now a guided setup screen at **Accounting → Payroll Setup (W-4)** in the admin sidebar — the same page that has held the payroll teaching material since books-13, with the working screen added above it. Pick an employee from the roster, and you get a checklist at the top of the page before you type anything at all. The checklist has a row for each thing that has to be true — W-4 on file, I-9 complete, documents examined, pay recorded, new hire reported to the state. Each row is either done, or it tells you what is missing.

The Save button stays greyed out until every row is clean. That is not a cosmetic touch. It is the whole point. You asked for exactly this:

"It should have a check list of task to be completed before it lets you save them to the system, and if a field is missing, it should highlight it so something can't silently fail me in some way."

When a field is missing, the field itself gets outlined on screen and the reason is printed directly underneath it, in a sentence, in English. Not an error code. Not a red asterisk. A sentence explaining what is missing and what goes wrong if it stays missing.

At the bottom of the screen there is a worked paycheck. It takes the numbers you just entered and shows you one complete pay period: gross pay, every deduction that comes out of the employee's check, every tax that comes out of Greenway's pocket, and the arithmetic for each line with the IRS or Washington citation that authorizes it. You asked me to show and teach you the formulas rather than just print answers. Every line shows its own work.

The three things this screen caught while I was building it

I want to walk through these because they are the argument for why the screen is worth trusting. Each one was a real defect in code I had already written and already believed was fine.

The Social Security numbers were being blanked out. The roster page shows each employee masked, as `XXX-XX-1234`, so you can tell people apart without the full number sitting on screen. To build that mask I passed the last four digits into the existing masking function. That function, sensibly, refuses to work on anything that is not a full nine-digit number — so it threw the four digits away and returned `XXX-XX-????` for every single employee on the list. The one part of the number the mask exists to preserve was the part it destroyed.

What matters here is how I found out. I did not find it by reading the code; the code looked correct, which is why I wrote it. I found it by actually running that function on four digits and looking at what came back. That is the habit this whole system is built on, and it is the habit your grandfather had that a software tool usually does not.

The hourly rate field was not being highlighted, and nothing noticed. This is the important one. In your Sage screenshots, every single employee's hourly rate reads `0.00`. Sage saved all of them. A person on the payroll with no wage.

So the hourly rate is arguably the most important field on this entire screen, and it has a test making sure that when it is empty, it gets highlighted. To check that test was real, I deliberately broke the screen — I deleted the code that highlights the hourly rate — and ran the tests to see if they would notice.

They did not. All of them passed.

The reason is worth understanding because it is subtle. The system keeps a list of fields that are *always* required, and the tests were checking that list. But the hourly rate is not always required — it is only required for an hourly employee, and Greenway also has you, on salary. So it is asked for *conditionally*, which means it was never on the list the tests were reading. The tests were checking a list that happened to exclude the most important field on the screen.

Fixed by changing the approach entirely: instead of reading a list of field names, the tests now feed the system realistic broken employees — an hourly employee with no wage, a salaried employee with no salary, a W-4 with no signature — and check that every complaint it produces points at a field that actually exists on screen. Then I re-broke the highlight, and this time the tests caught it.

There was a live door into the system that nothing was using. I had written a piece of server code called `checkSetupAction` — a way for the screen to ask the server "is this employee ready to save?" It was properly permission-gated, it was tested, and it looked like solid work.

Nothing called it. Not one line anywhere in the application. The screen already worked out the checklist on its own as you type, which is what makes the guidance appear immediately instead of after you press Save, so the server version had never been needed.

Unused code is usually just clutter. This was worse, for two reasons. First, it was a real, reachable, authenticated entry point into your payroll data that no screen exercised and nobody would think to review. Second, and worse for you specifically: it was a *second* place that answered the question "is this employee ready?" Two answers to one question is how you end up with two screens disagreeing about the same employee, which is the exact failure this slice was built to prevent. I deleted it, and added a permanent test that refuses to let any future server action exist without something actually calling it.

A fourth one, found by proofreading this very report

There is a rule I follow that says the write-up I hand you has to be checked against the code, line by line, rather than written from memory. It exists because a report that describes software that does not exist is worse than no report at all — you would make decisions on it.

Doing that check on this document, I got to my own sentence claiming every paycheck line shows the IRS or Washington citation that authorizes it, and went to confirm it. Each line did carry its citation internally. The screen was throwing it away and printing only the arithmetic.

So the paycheck was showing you `$1,440.00 × 6.2% = $89.28` with nothing behind it. That is a number somebody typed. 26 U.S.C. §3101(a) is the *reason* it is 6.2%, and being able to follow a figure back to the rule behind it is the entire difference between this system and a spreadsheet that agrees with itself.

Now fixed — the citation prints under each line, looked up from the shared list of authorities rather than typed next to the number. That distinction matters more than it sounds: a citation typed by hand next to a computed figure looks completely authoritative and goes stale the moment the rule changes, with nothing to flag it. A looked-up one either resolves to the real rule or shows nothing at all. There are now tests for both — one that the citation reaches the page, and one that every authority the paycheck references actually exists.

I mention it because it is a good illustration of why I write these to you in plain English rather than just shipping the code. Explaining the system to you is how I find out what the system does not actually do.

The part I am least comfortable with, and what I did about it

The tests that guard this screen use example employees — a complete, correct employee, and then deliberately broken copies of that employee. Standard practice.

I decided to attack the examples instead of the code, on the theory that if the examples are wrong, everything built on them is decoration. So I made one tiny change: on the "complete, correct" employee, I removed the W-4 signature.

Every test still passed. All thirty-seven of them.

Here is what had happened. Every broken example was made by copying the complete one and breaking a single thing. Once the original was missing a signature, each copy was broken in *two* ways — the thing under test, plus the missing signature. And every test only asked "was this refused?" The answer was still yes, so everything stayed green. Meanwhile three separate tests — the hourly rate highlight, the salary highlight, and the signature check — had all quietly stopped testing anything at all.

One character of damage to an example, and three tests silently went hollow. Nothing turned red.

This is fixed in two ways. The tests now check that the "complete" employee really is complete — that the system accepts it with zero complaints — because "this employee is correct" is itself a claim worth verifying. And each broken example now has to be refused for *exactly* its one intended reason, not merely refused for some reason. Then I proved that second change earns its keep by damaging only one of the copies; the old style of test missed it and the new one caught it by name.

I ran fourteen of these deliberate-sabotage experiments against this screen. Twelve were caught immediately. Two were not — the hourly rate highlight and this example problem. Both are now fixed and both are now caught.

One field is deliberately left blank

The pay section asks for the Washington minimum wage in effect on the hire date. That box ships empty, with a note explaining why.

I went looking for the minimum wage in the system's rate tables, expecting to find it and fill the field in automatically. It is not there. The rate file has Labor & Industries rates, paid family leave rates, unemployment rates, the Social Security wage base — all sourced and documented — and no minimum wage.

I could have typed in a number I found on the internet. I would rather the screen tell you it does not know than hand you a legal wage floor that nobody verified. A wrong minimum wage does not look wrong; it looks like a fact. When we do the company-wide payroll setup in the next slice, that figure gets sourced properly from the Department of Labor & Industries and cited like everything else.

Where this leaves you

Practically: the screen exists and works, but it cannot save anything until you apply the database change. That is on the list below.

Bigger picture — you said the purpose of this system is to replace your grandfather with something that both protects you and teaches you. The part of that I keep coming back to is that your grandfather would have *told you why*. Not just "this is wrong," but "this is wrong, and here is what happens if you file it that way." That is what the sentences under each field are for, and it is why the worked paycheck shows arithmetic instead of just totals.

Four defects found in a few hours of work on a single screen. Every one of them would have been invisible in Sage; two of them were invisible in my own tests until I went looking for them on purpose, and the fourth only surfaced because I had to explain the screen to you in writing. That is the difference between software that works and software you can rely on when the stakes are your own tax return.

What I need from you

Apply the database migration. In the Supabase SQL editor, run `0195_employee_payroll_setup.sql` , and also `0193_security_log_owner_only.sql` and `0194_cycle_counts_read_only.sql` if you have not already. After 0195, run this to confirm it landed correctly:

```
select * from public.gl_audit_employee_payroll_setup();
```

An empty result means everything is right. Any rows returned means something needs looking at, and each row will say what. Until 0195 is applied the setup screen shows a plain notice explaining that it cannot save yet, rather than letting you fill in a form that would fail at the end.

Still outstanding from earlier slices — I list these every time so they do not get lost: the exact tax year of your S-election, the ending Accumulated Adjustments Account balance from Schedule M-2, the per-shareholder penalty amount under §6699(e), the quarterly federal short-term interest rates, your depreciation schedule and asset list, your grandfather's work papers, the twelve years of returns, and the intercompany detail.

And when you get a chance — you offered to send an example of how Sage structures its report data. I would like that before books-27, which is the reports engine. You told me you do not use the Sage reports because you cannot fully tell what they are showing you, and I am treating that as a design requirement rather than a complaint: a report you cannot read is a report that does not exist.

What comes next

books-26 is the company side of payroll setup — Greenway's own tax accounts, deposit schedule, the state rate assignments, and the properly sourced minimum wage. Then books-27 is the reports engine, built so that every number on every report can

be traced back to the transaction it came from.

The migration question you asked about — how we move you from your current setup onto this one for the January 1, 2027 first payroll — I owe you an answer on that at the end of this slice, and I have not forgotten it. I want the employee and company setup screens both finished before I commit to a plan, because the plan depends on exactly what has to be re-entered by hand and what can be carried over.